

Foundations of Retrieval-Augmented Generation

How AI assistants answer from enterprise data: embeddings, vector search and grounded responses

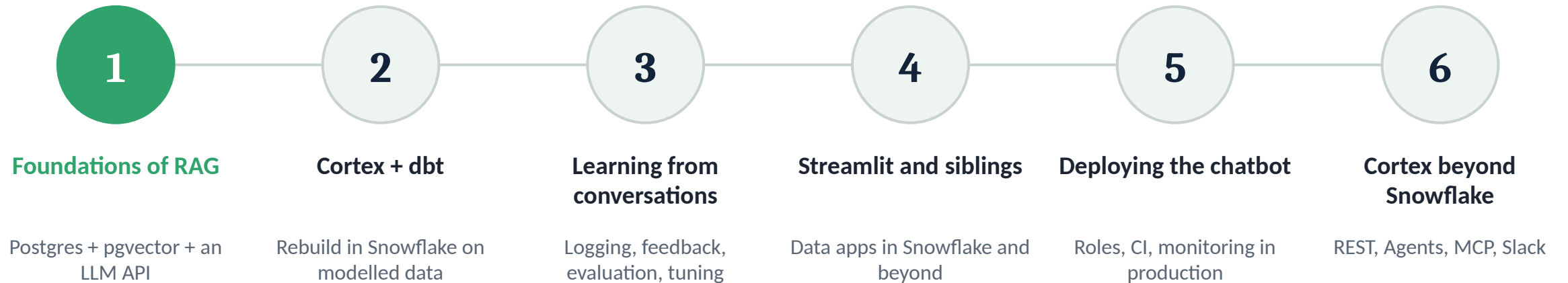
R. Firman Insan Muhammad (Vim)

SnowPro Certified · Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



One chatbot, six iterations



Today's output: a working baseline chatbot and a small golden test set. Every later module is measured against it.

By the end of today you can...



Explain

why RAG exists: parametric vs non-parametric knowledge, and the failure modes retrieval addresses.



Reason

about embeddings, similarity metrics and chunking as design decisions with trade-offs, not library defaults.



Compare

ANN index families (HNSW, IVF, PQ) and choose a vector store for a given workload and organisation.



Build and critique

an end-to-end RAG chatbot over Postgres + pgvector, and measure its retrieval quality.

An LLM alone cannot answer questions about your business



Stale

Knowledge freezes at the training cutoff. Last week's reward change or a member's points balance simply does not exist for the model.



Blind

Your tables, documents and tickets were never in the training data, and should not be. The model has no access path to them.



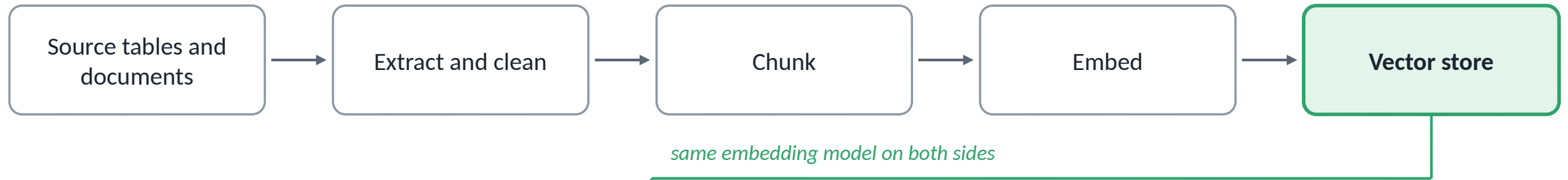
Unaccountable

Answers arrive fluent and confident with no provenance. You cannot tell recall from invention, and neither can a regulator.

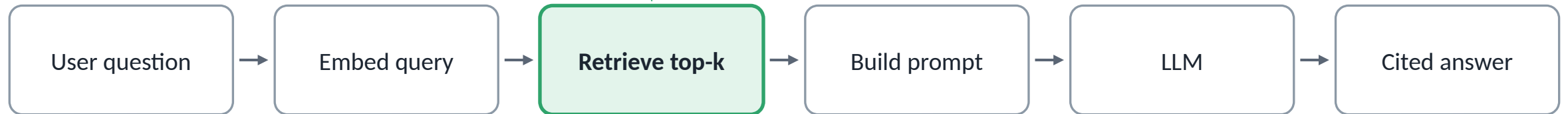
RAG in one sentence: *give the model an open book at inference time, and make it show which page it read.*

RAG is two pipelines that share one index

OFFLINE • INDEXING (BATCH OR CDC)



ONLINE • PER QUESTION (LATENCY-SENSITIVE)



Data engineering owns the top lane. Most production RAG failures start there.

Where RAG came from



Today's lab is "naive RAG" in the survey's taxonomy. Modules 2 and 3 move us to advanced RAG.

01

Representing meaning

Embeddings, similarity and chunking: the three decisions that fix retrieval quality before any LLM is called

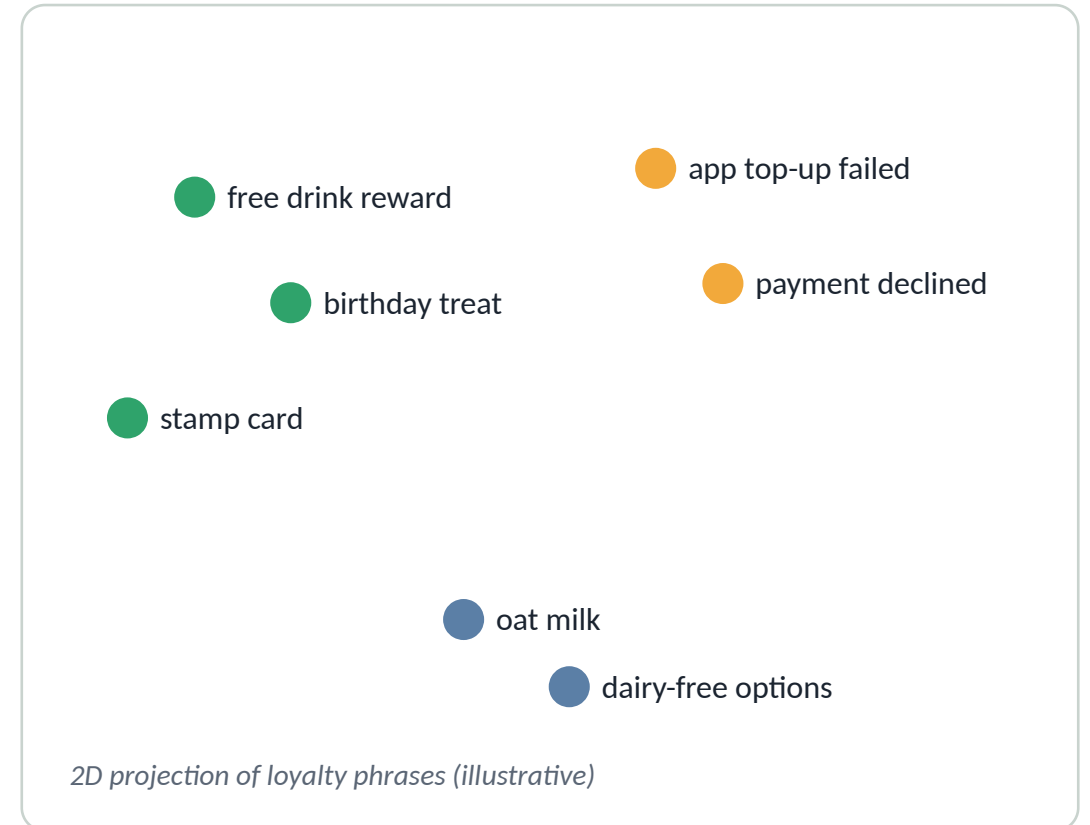
Text becomes a point; meaning becomes distance

An embedding model f maps text to a vector in $\mathbb{R}^d$ so that similar meaning implies nearby vectors.

$$\cos(q, d) = (q \cdot d) / (\|q\| \|d\|)$$

Metric	Measures	pgvector op
Cosine distance	Angle only	<=>
Inner product	Angle and magnitude	<#>
Euclidean (L2)	Straight-line distance	<->

Normalise to unit length and all three rank identically.



Choose the model on your queries, not the leaderboard

Model	Dims	Max tokens	Why consider it
all-MiniLM-L6-v2	384	256	Tiny, fast English baseline
BAAI/bge-small-en-v1.5	384	512	Today's lab default: strong for its size
BAAI/bge-m3	1024	8192	Multilingual; dense and sparse outputs
OpenAI text-embedding-3-small	1536	8191	Hosted API; dimensions can be shortened
Snowflake arctic-embed-m-v1.5	768	512	Open weights; available inside Cortex (L2)

Rule: never mix embedding models in one index. The model is part of the schema; changing it means re-embedding everything.

Selection criteria

- Languages in your corpus and queries
- Domain vocabulary (menu, loyalty terms)
- Max input vs your chunk size
- Dimensions: storage and latency cost
- Licence and data residency
- Recall on your own golden set

The most underrated decision in the pipeline

Fixed-size + overlap

Every N tokens, with an overlap window

amber = overlap



Structural (recursive)

Split on headings, then paragraphs, then sentences



Semantic

Split where adjacent-sentence similarity drops



Parent-child

Match small chunks, return their parent section



Too small: precise hits, missing context. **Too large:** diluted embedding, wasted prompt tokens.

Starting point: 300 to 800 tokens, 10 to 20% overlap, respect document structure. Then measure.

02

Vector indexes and stores

Why exact search does not scale, how approximate nearest-neighbour indexes work, and where to keep your vectors

Exact search is linear; production needs approximate

$O(n \cdot d)$

per query for exact search

10M chunks $\times$ 768 dims $\approx$ 7.7 billion
multiply-adds for every single question



HNSW (graph)

Navigable multi-layer graph. High recall, fast queries, incremental inserts. Memory-hungry.



IVF (clustering)

k-means partitions; search the nprobe nearest lists. Needs training; suits large, stable sets.



PQ (compression)

Split vectors into sub-vectors and store codes. Big memory savings, lower recall. Often IVF-PQ.

Skip-list thinking in high-dimensional space



Parameter	Trades	pgvector default
m	Links per node: memory vs recall	16
ef_construction	Build time vs graph quality	64
ef_search	Query latency vs recall	40

Greedy descent: find the nearest node on each layer, then drop down. Tune ef_search per query, not per index.

Extend the database you run before adding a new one



Extension of an existing DB

pgvector, Snowflake VECTOR,
Elasticsearch / OpenSearch, MongoDB
Atlas

*Pick when data already lives there and
governance matters*



Purpose-built vector DB

Pinecone, Qdrant, Weaviate, Milvus

*Pick at very large scale or high QPS, or for
their advanced features*



Library you embed

FAISS, hnswlib, ScaNN

*Pick for research, benchmarks, or when you
own serving end to end*

Data engineer's lens: freshness, joins with metadata, one access-control model, one fewer system to run.

03

Reading from the database

Two kinds of question, two retrieval routes, and the lab schema that ties them together

Similarity search cannot count



Semantic: RAG over text

"Can members redeem a free drink on a delivery order?"

Sources: programme terms, rewards FAQs, store guides

Mechanism: embed, retrieve top-k, ground the answer



router



Analytical: text-to-SQL

"How many free-drink rewards were redeemed in London in Q2?"

Sources: members, rewards, transactions tables

Mechanism: generate SQL over a modelled schema, execute, explain

Module 2 preview: Cortex Search is the managed left route, Cortex Analyst the managed right route, both fed by dbt models.

Keep chunks beside their source key

POSTGRES 16 + PGVECTOR

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE doc_chunks (
  id          BIGSERIAL PRIMARY KEY,
  doc_id      TEXT NOT NULL,
  chunk_no    INT  NOT NULL,
  chunk_text  TEXT NOT NULL,
  metadata    JSONB,    -- channel, section, model
  embedding   VECTOR(384),
  updated_at  TIMESTAMPTZ DEFAULT now(),
  UNIQUE (doc_id, chunk_no)
);

CREATE INDEX ON doc_chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

CREATE INDEX ON doc_chunks USING gin (metadata);
```

Lineage

doc_id ties every chunk to its source row or file, so deletes and updates propagate.

Idempotency

(doc_id, chunk_no) is the upsert key; reruns never duplicate.

Pre-filtering

metadata carries channel, section and model version for WHERE clauses.

Fixed dimension

384 is set by the embedding model; a new model means a new column or table.

Ingestion: chunk, embed, upsert

INGEST.PY

```
from sentence_transformers import SentenceTransformer
from pgvector.psycopg import register_vector
from psycopg.types.json import Jsonb
import psycopg

MODEL = "BAAI/bge-small-en-v1.5"          # 384 dims
model = SentenceTransformer(MODEL)

def chunk(text, size=800, overlap=150):
    step = size - overlap
    return [text[i:i + size] for i in range(0, len(text), step)]

UPSERT = """INSERT INTO doc_chunks
(doc_id, chunk_no, chunk_text, metadata, embedding)
VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (doc_id, chunk_no) DO UPDATE
SET chunk_text = EXCLUDED.chunk_text,
    embedding = EXCLUDED.embedding, updated_at = now()"""

with psycopg.connect(DSN) as conn:
    register_vector(conn)
    for doc in load_loyalty_docs():          # id, text, meta
        parts = chunk(doc.text)
        vecs = model.encode(parts, normalize_embeddings=True)
        for i, (t, v) in enumerate(zip(parts, vecs)):
            conn.execute(UPSERT, (doc.id, i, t,
                                   Jsonb(**doc.meta, "model": MODEL}), v))
```

Production gaps to close

- Chunk by structure, not characters
- Batch inserts; do not commit per row
- Delete orphaned chunks when a document shrinks
- Re-embed only changed documents
- Log counts and failures per run

Retrieve, then generate with citations

RETRIEVE: RAG.PY

```
def retrieve(question, k=5, channel=None):
    qv = model.encode(question,
                        normalize_embeddings=True)
    return conn.execute("""
        SELECT doc_id, chunk_no, chunk_text,
               1 - (embedding <=> %s) AS score
        FROM doc_chunks
        WHERE %s::text IS NULL
           OR metadata->>'channel' = %s
        ORDER BY embedding <=> %s
        LIMIT %s""",
        (qv, channel, channel, qv, k)).fetchall()
```

GENERATE: RAG.PY

```
SYSTEM = """You are Neo Nexus, the loyalty
assistant. Answer ONLY from the
context. Cite sources as [doc_id#chunk].
If the context lacks the answer, say so."""

def answer(question, channel=None):
    hits = retrieve(question, 5, channel)
    context = "\n\n".join(
        f"[{h[0]},{h[1]}] {h[2]}" for h in hits)
    user = f"Context:\n{context}\n\n" \
          f"Question: {question}"
    return llm_chat(SYSTEM, user, hits)
```

Same model, both sides

The query is embedded exactly as the chunks were, normalised.

IDs make answers checkable

Every citation maps back to one row in doc_chunks.

Abstaining beats guessing

An honest "I don't know" is a correct answer.

One interface, any engine: Claude, GPT or Llama

LLM.PY

```
# llm.py: rag.py only ever calls llm_chat(system, user)
def llm_chat(system, user, engine="claude"):
    if engine == "claude":                # Anthropic API
        import anthropic
        r = anthropic.Anthropic().messages.create(
            model="claude-sonnet-5-5", max_tokens=1024,
            system=system,
            messages=[{"role": "user", "content": user}])
        return r.content[0].text
    if engine == "openai":                # OpenAI API
        from openai import OpenAI
        r = OpenAI().chat.completions.create(
            model="gpt-5-mini",
            messages=[{"role": "system", "content": system},
                      {"role": "user", "content": user}])
        return r.choices[0].message.content
    if engine == "llama":                 # local, via Ollama
        import ollama
        r = ollama.chat(model="llama3.3:70b", messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": user}])
        return r["message"]["content"]
```

Anthropic Claude

API key · data goes to Anthropic

OpenAI GPT

API key · data goes to OpenAI

Meta Llama

open weights · runs on your hardware

Design rule: keep the engine behind one function so it can be swapped and compared.

Size the engine to the job, then check where the data goes

Engine family	Tiers (large → small)	How you access it	Where the prompt goes	Typical fit
Anthropic Claude	Opus → Sonnet → Haiku	Anthropic API; major clouds	Anthropic, or your cloud provider	Long documents, careful reasoning
OpenAI GPT	GPT-5 → mini → nano	OpenAI API; Azure OpenAI	OpenAI or Azure	Broad ecosystem and tooling
Meta Llama	70B → 8B	Open weights: Ollama, vLLM, clouds	Can stay on your hardware	Privacy, offline, cost at scale
Mistral	Large → small	API or open weights	Vendor or self-hosted	European hosting, efficient models

Small, fast

routing, classification, query rewriting

Mid-tier

grounded answers from retrieved context

Largest

hard reasoning, evaluation judging

For RAG, context quality usually matters more than engine size. Measure on your golden set before paying for the biggest model.

The prompt is a contract, not a wish

Role and rules

Who the assistant is; answer only from context

Retrieved context

Chunks, each tagged with a stable ID

User question

Verbatim, after the context

Output contract

Cite IDs; abstain if unsupported; format

Lost in the middle

Models use the start and end of long contexts best. Rank the strongest chunks first and keep k small.

Context budget

Beyond a few relevant chunks, extra context adds cost and distraction, not accuracy.

Indirect prompt injection

Retrieved text is untrusted input. A document can contain instructions; treat it as data.

Four upgrades, each fixing a named failure



Hybrid search

Fixes: exact terms (reward codes, clause numbers) lost by dense vectors. BM25 + vector, fused with Reciprocal Rank Fusion.



Reranking

Fixes: plausible but wrong top hits. A cross-encoder rescores the top 50 and keeps the best 5.



Query transformation

Fixes: vague or multi-part questions. Rewrite, split into sub-queries, or embed a hypothetical answer (HyDE).



Structure and metadata

Fixes: right topic, wrong channel or version. Pre-filter, parent-child retrieval, graph-based retrieval.

If you cannot measure retrieval, you are guessing



Retrieval quality

Recall@k: is a correct chunk in the top k?

MRR: how high is the first correct chunk?

nDCG: graded relevance, discounted by rank

Needs labelled question-to-chunk pairs



Answer quality

Faithfulness: is every claim supported by context?

Answer relevance: does it address the question?

Context precision: was the retrieved context useful?

Usually LLM-as-judge with a rubric (e.g. RAGAS)

Today: a 20-question golden set and recall@5. **Module 3:** the full evaluation and feedback loop.

Build the Neo Nexus loyalty assistant

- 1 Start Postgres + pgvector**
docker run pgvector/pgvector:pg16; create schema
10 min
- 2 Load sample data**
loyalty tables and 12 programme documents
10 min
- 3 Chunk and embed twice**
two configs: 400 and 1,000 characters
25 min
- 4 Retrieve and answer**
rag.py with citations and an abstain rule
25 min
- 5 Evaluate**
20-question golden set; recall@5 per config
20 min

Deliverables

- Repo: ingest.py, rag.py, eval notebook
- Recall@5 for both chunk sizes
- Three failure cases, each diagnosed: retrieval or generation?
- One-page reflection: which chunk size, and why

Stretch: structural chunking, or BM25 + RRF hybrid

Questions worth arguing about

1. With million-token context windows, when is retrieval still worth building?
2. Is cosine similarity a good proxy for relevance? Construct a counter-example.
3. Where should access control live in a RAG system: the index, the query, or the prompt?
4. What does freshness mean for an index when the source table changes hourly?



Reading list

Lewis et al. (2020) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

Karpukhin et al. (2020) Dense Passage Retrieval for Open-Domain QA. EMNLP.

Malkov and Yashunin (2018) Efficient and robust ANN search using HNSW graphs. IEEE TPAMI.

Liu et al. (2023) Lost in the Middle: How Language Models Use Long Contexts.

Gao et al. (2023) Retrieval-Augmented Generation for LLMs: A Survey.

Es et al. (2023) RAGAS: Automated Evaluation of RAG.

Same chatbot, rebuilt inside Snowflake



dbt layer

Staging and mart models, tested and documented, instead of raw tables



Cortex Search

Replaces today's embedding code, index and hybrid search



Cortex Analyst

Answers the analytical questions via a semantic model

Before Module 2: finish the lab, read Lewis et al. (2020) and Liu et al. (2023), and create a Snowflake trial account in a Cortex-enabled region.